

GyaanSetu Placement Interview Master Guide

Complete Comprehensive Question Bank, Architecture Defense, Database ER Diagrams, Visual Revision, 50+ Rapid Fire Q&A & 60-Sec Speeches

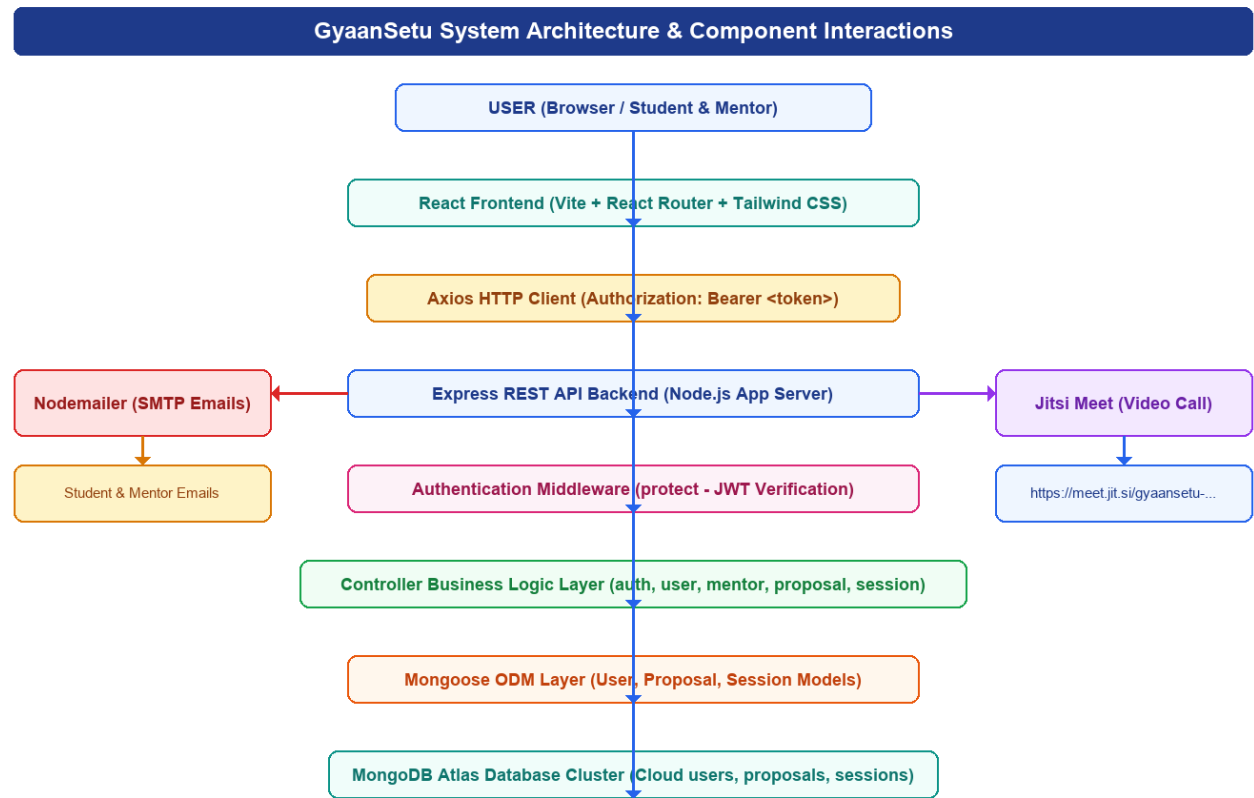
Author / Candidate:	Placement Candidate (Aditya)
Project Name:	GyaanSetu – Mentor-Student Doubt Solving Platform
Document Purpose:	Complete placement interview preparation covering 16 detailed question categories
Includes:	Embedded Visual Diagrams, 50+ Rapid Fire Q&A, Mock Interview Transcript & 60-Sec Pitches
Target Placement:	Software Engineer / SDE-1 / Full-Stack MERN Developer

Table of Contents (16 Categories + Visual Revision)

SPECIAL: Architecture & Database Visual Revision	Cat 8. Single Account Dual Role Architecture
Cat 1. Project Introduction & Overview	Cat 9. Session Scheduling & Conflicts
Cat 2. Core MERN Architecture & 8 Layers	Cat 10. Email Notifications & Nodemailer
Cat 3. Database Design, Collections & ER	Cat 11. Security & Ownership Authorization
Cat 4. Authentication (bcrypt & JWT)	Cat 12. Edge Cases & Error Handling
Cat 5. Backend (Node.js & Express.js)	Cat 13. 'Why Did You Use This?' Selection
Cat 6. Frontend (React 18 & Hooks)	Cat 14. Real Challenges & Future Scope
Cat 7. Complete API Endpoint Questions	Cat 15. 50+ Rapid Fire Revision Round & Speeches

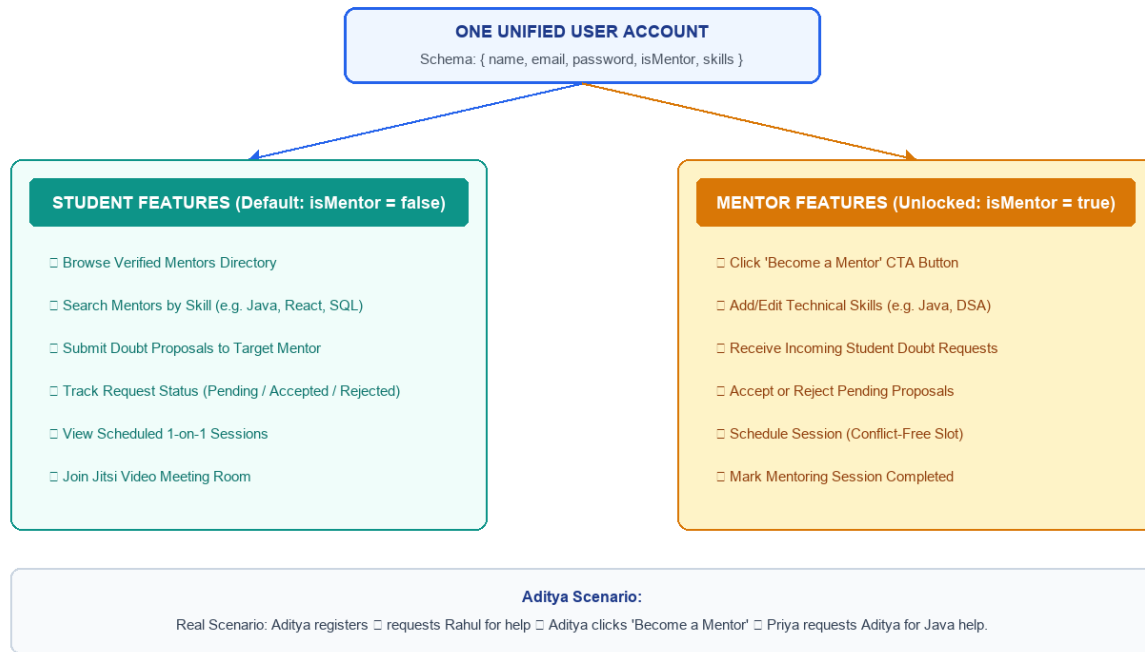
Architecture & Database — Visual Revision

Quick 2-minute visual revision before an interview to review the system components and database relationships.



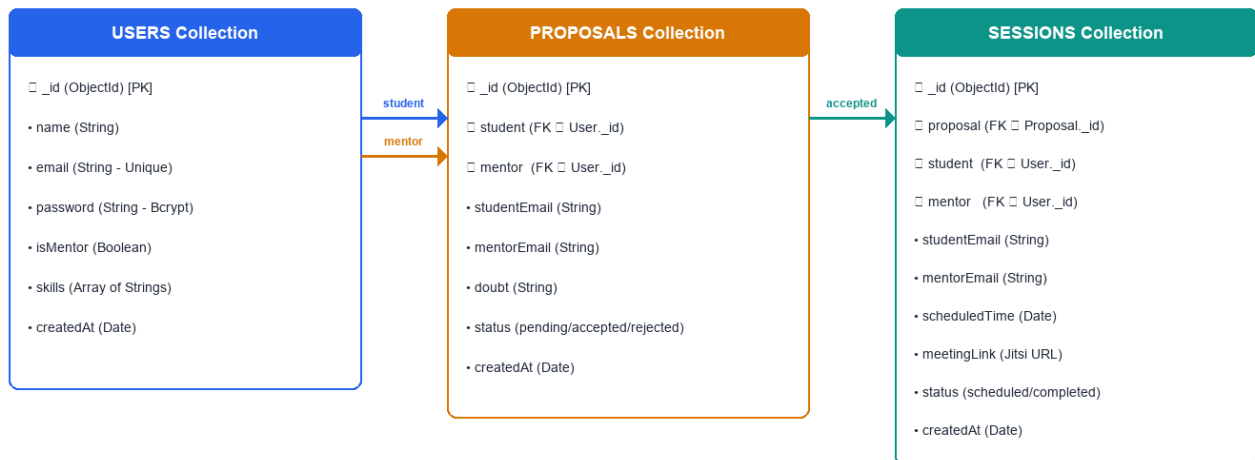
Visual Revision 1: System Architecture & Ecosystem

Single Account Architecture — Student & Mentor Dual Capabilities



Visual Revision 2: Single Account Dual Role Capabilities

GyaanSetu MongoDB Atlas ER Database Schema Diagram



Mongoose ObjectId References & Integrity Rules:

1. proposal.student & proposal.mentor store User ObjectIds (Mongoose populate('student mentor')).
2. session.proposal stores Proposal ObjectId (links accepted doubt to scheduled time & Jitsi room).
3. Decoupling rationale: Keeps user documents small and avoids 16MB document size limit.

Visual Revision 3: MongoDB Atlas ER Schema & References

Category 1 — Project Introduction & Overview

Q1: Tell me about your GyaanSetu project.

Interview Answer: GyaanSetu is a web-based mentorship and doubt solving platform built using the MERN stack (MongoDB, Express.js, React.js, Node.js). It replaces informal messaging by connecting students with technical mentors for structured 1-on-1 doubt solving sessions. A key architectural highlight is its single-account model: every user starts as a student, but any user can click 'Become a Mentor' to set `isMentor: true` and add skills. The same account can search for mentors, request help, and ALSO receive, accept, and schedule sessions for incoming student doubts.

Q2: What problem does GyaanSetu solve?

Interview Answer: Students usually ask technical doubts via informal chat apps like WhatsApp or Telegram where requests get buried, schedule conflicts occur, and there is no dedicated environment for code review. GyaanSetu provides a structured workflow: doubt request submit -> mentor accept -> conflict-free scheduling -> instant Jitsi video meeting link generation -> email notification.

Category 2 — Core MERN Architecture & 8 Layers

Q: Explain the request-response lifecycle of your application.

Answer: When a user clicks an action in React, an event handler calls our central Axios instance (`api.js`). The request interceptor automatically attaches the user's JWT token in the `Authorization: Bearer `` header. The Express server routes the request to an API endpoint (`/api/...`), passing through `protect`` middleware which verifies the JWT signature and populates `req.user``. The controller executes business logic, queries MongoDB Atlas via Mongoose models, and returns JSON. React receives the JSON response and updates state, causing the UI to re-render dynamically.

Category 3 — Database Design, Collections & ER Diagram

Q: Explain your MongoDB collections, schemas, and relationships.

Answer: We have 3 normalized collections inside MongoDB Atlas:

1. `users``: Stores `name``, `email`` (unique), `password`` (hashed), `isMentor`` (boolean), `skills`` (string array).
2. `proposals``: Stores `student`` (ref User), `mentor`` (ref User), `doubt``, and `status`` (`pending``, `accepted``, `rejected``).
3. `sessions``: Stores `proposal`` (ref Proposal), `student`` (ref User), `mentor`` (ref User), `scheduledTime``, `meetingLink``, and `status`` (`scheduled``, `completed``).

References allow Mongoose to execute `.populate('student mentor')`` to perform join-like data enrichment.

Category 4 — Authentication (bcrypt & JWT)

Q: How is authentication implemented in GyaanSetu? Why bcrypt and JWT?

Answer: Passwords are never saved in plain text. During registration, `bcrypt.hash(password, 10)`` generates a salt and hashes the password. During login, `bcrypt.compare()`` checks the password hash. If valid, `jwt.sign({ userId, email })`` issues a 30-day signed token. The frontend stores this token in `localStorage`` and includes it in the `Authorization`` header for protected endpoints.

Category 5 — Node.js & Express.js Backend

Q: What is middleware in Express and how is it used in your project?

Answer: Middleware functions have access to `req`, `res`, and `next`. In GyaanSetu:

- Built-in middleware like `express.json()` parses JSON payloads.
- Third-party middleware like `cors()` enables cross-origin browser requests.
- Custom middleware `protect` verifies JWT tokens before allowing access to protected controllers.

Category 6 — React 18 & Frontend Architecture

Q: How does state management work in your React frontend?

Answer: We use React Context (`AuthContext`) for global user identity state (`user`, `token`, `login`, `logout`).

Component-level state (like form inputs, search terms, modal visibility) is managed using `useState`. Asynchronous API fetches are triggered on component mount using `useEffect`.

Category 7 — Complete API Endpoint Reference Q&A

Q: Walk through POST `/api/proposals` step by step.

1. Request Body: `{ mentorId, doubt }` + Bearer JWT token.
2. Middleware verifies JWT, attaches `req.user`.
3. Controller validates `req.user._id !== mentorId` (prevents self-requesting).
4. Controller checks `Proposal.findOne({ student, mentor, status: 'pending' })` (prevents duplicate requests).
5. Creates proposal document in MongoDB Atlas and returns `201 Created`.

Category 8 — Single Account Dual Role Architecture

Q: Why did you choose ONE User model instead of separate Student and Mentor models?

Answer: In real college environments, senior students are mentors for some topics while still being learners for other topics. Forcing separate logins forces users to register two accounts with different emails. By using a single `User` model with `isMentor: Boolean`, every account can act as a student, and enabling `isMentor = true` grants additional mentor capabilities without splitting user identity.

Category 9 & 10 — Scheduling, Conflicts & Nodemailer

Q: How do you prevent time slot conflicts?

Answer: Before creating a session, `sessionController.createSession` queries `Session` for any existing scheduled session for either the student or mentor within a +/- 29 minute window around `scheduledTime`. If a match is found, it returns HTTP `400` Time slot is already occupied.

Category 11 & 12 — Security Authorization & Edge Cases

Q: How do you ensure a user cannot accept another mentor's proposal?

Answer: Ownership validation! When ``PATCH /api/proposals/:id/accept`` is called, the controller fetches the proposal and checks ``if (proposal.mentor.toString() !== req.user._id.toString()) return res.status(403).json({ message: 'Unauthorized action.' })``.

Category 13 — 'Why Did You Use This?' Technology Selection

- **Why React over Plain JS?** Component reusability, virtual DOM diffing, declarative UI state management.
- **Why Vite over Create-React-App?** Instant dev server boot up, lightning fast HMR, and small production bundle size.
- **Why MongoDB over SQL?** Schema flexibility for skill array tags and rapid MERN development.
- **Why Jitsi Meet?** Open-source video meeting rooms generated via simple URLs without complex WebRTC backend signaling infrastructure.

Category 14 — Project Challenges & Future Scope

- **Challenge Encountered:** Handling DNS SRV record lookup errors (``querySrv ECONNREFUSED``) on restricted Wi-Fi networks when connecting to MongoDB Atlas.
- **Solution:** Configured ``dns.setServers(['8.8.8.8', '8.8.4.4'])`` in Node.js to use Google Public DNS.
- **Future Version 2 Scope:** Real-time socket.io chat during live meetings, rating & review feedback scores for mentors, and code snippet collaborative editor.

Category 15 — 50 Rapid Fire Interview Q&A Round

Question	Concise 1-Line Answer
1. What is JWT?	JSON Web Token - a compact, URL-safe cryptographic token used for stateless authentication.
2. What is bcrypt?	A password-hashing function using salt to securely hash passwords before storing in DB.
3. What is Mongoose?	An Object Data Modeling (ODM) library for MongoDB and Node.js.
4. What is CORS?	Cross-Origin Resource Sharing - HTTP mechanism allowing frontend (port 5173) to access backend API (port 5000).
5. What is REST?	Representational State Transfer - standard stateless API architecture using standard HTTP verbs.
6. What is <code>`isMentor`</code> ?	Boolean field on User model enabling mentor directory visibility and dashboard controls.
7. What is <code>`req.body`</code> ?	Contains key-value pairs of data submitted in HTTP POST/PATCH request body.
8. What is <code>`req.params`</code> ?	Contains URL route parameters (e.g. <code>`:id`</code> in <code>`/api/proposals/:id/accept`</code>).
9. What is <code>`req.user`</code> ?	Authenticated user object attached to request object by <code>`protect`</code> middleware.
10. What is <code>`populate()`</code> ?	Mongoose query method replacing reference ObjectIds with actual referenced document data.

Category 16 — 60-Second Architecture & Database Speeches

EXPLAIN MY PROJECT ARCHITECTURE IN 60 SECONDS:

"GyaanSetu is built on a decoupled MERN architecture. The frontend is a React 18 SPA compiled with Vite and styled using Tailwind CSS. It communicates statelessly with an Express REST API running on Node.js using Axios. Requests carry JWT tokens in the `Authorization: Bearer` header, which custom `protect` middleware verifies to populate `req.user`. Controllers execute business rules, validation, and conflict checks before persisting data into MongoDB Atlas via Mongoose ODM. We also integrate Nodemailer for session email alerts and Jitsi Meet for automated 1-on-1 video call rooms. The architecture enforces a single-account model where any student can upgrade to a mentor without separate logins."

EXPLAIN MY DATABASE DESIGN IN 60 SECONDS:

"Our MongoDB Atlas database contains 3 normalized collections: `users`, `proposals`, and `sessions`. The `users` collection stores user details, hashed passwords, `isMentor` boolean flag, and skills array. The `proposals` collection links a student `ObjectId` to a mentor `ObjectId` along with the doubt description and status state machine (`pending`, `accepted`, `rejected`). The `sessions` collection links an accepted proposal to a scheduled time, status (`scheduled`, `completed`), and a generated Jitsi meeting URL. We use Mongoose `ObjectId` references to perform populated queries while avoiding unbounded array growth inside user documents."